Delete Data - DELETE

Now let's delete some data using **SQLModel**.

Continue From Previous Code

As before, we'll continue from where we left off with the previous code.

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero_1 = results.one()
        print("Hero 1:", hero_1)

        statement = select(Hero).where(Hero.name == "Captain North America")
        results = session.exec(statement)
        hero_2 = results.one()
        print("Hero 2:", hero_2)

        hero_1.age = 16
        hero_1.name = "Spider-Youngster"
        session.add(hero_1)

        hero_2.name = "Captain North America Except Canada"
        hero_2.age = 110
        session.add(hero_2)

        session.commit()
        session.refresh(hero_1)
        session.refresh(hero_2)

        print("Updated hero 1:", hero_1)
        print("Updated hero 2:", hero_2)


def main():
```

```
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Remember to remove the `database.db` file before running the examples to get the same results.

Delete with SQL

This `Spider-Youngster` is getting too weird, so let's just delete it.

But don't worry, we'll reboot it later with a new story. 😅

Let's see how to delete it with **SQL**:

```
DELETE
FROM hero
WHERE name = "Spider-Youngster"
```

This means, more or less:

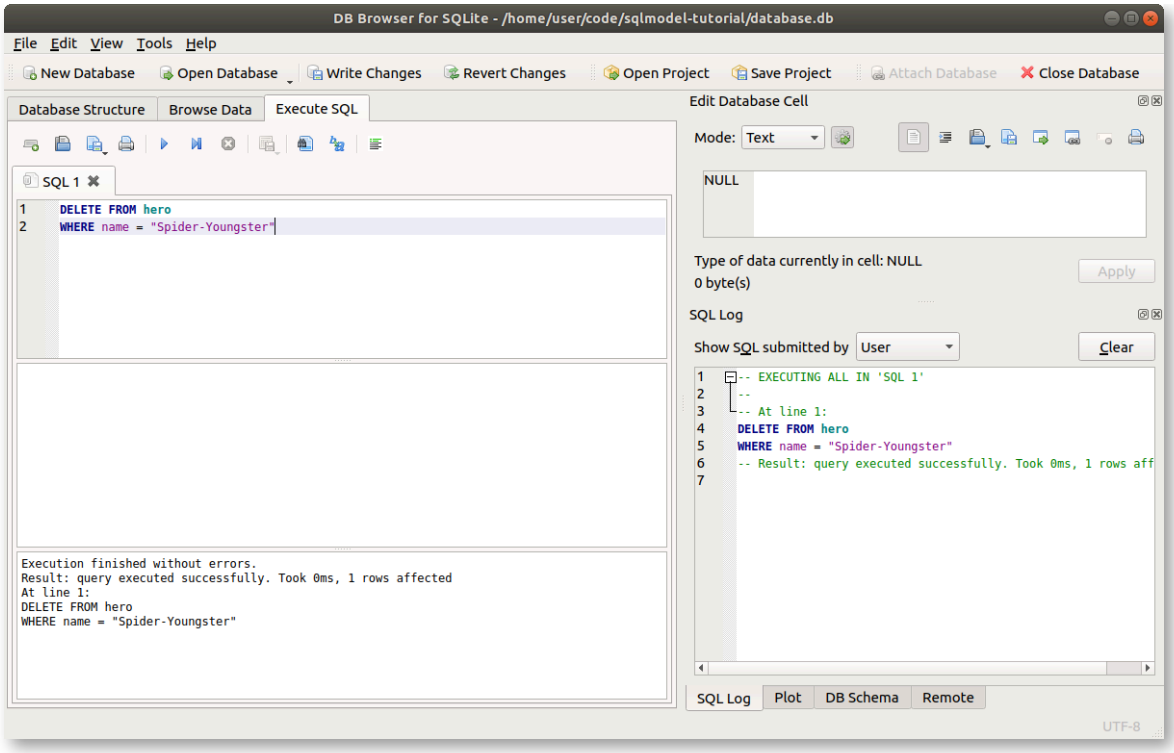
> Hey SQL database 👋, I want to `DELETE` rows `FROM` the table called `hero`.
>
> Please delete all the rows `WHERE` the value of the column `name` is equal to `"Spider-Youngster"`.

Remember that when using a `SELECT` statement it has the form:

```
SELECT [some stuff here]
FROM [name of a table here]
WHERE [some condition here]
```

`DELETE` is very similar, and again we use `FROM` to tell the table to work on, and we use `WHERE` to tell the condition to use to match the rows that we want to delete.

You can try that in **DB Browser for SQLite**:



Have in mind that `DELETE` is to delete entire **rows**, not single values in a row.

If you want to "delete" a single value in a column while **keeping the row**, you would instead **update** the row as explained in the previous chapter, setting the specific value of the column in that row to `NULL` (to `None` in Python).

Now let's delete with **SQLModel**.

To get the same results, delete the `database.db` file before running the examples.

Read From the Database

We'll start by selecting the hero `"Spider-Youngster"` that we updated in the previous chapter, this is the one we will delete:

Python 3.10+

```
# Code above omitted 👆

def delete_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.one()
        print("Hero: ", hero)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero_1 = results.one()
        print("Hero 1:", hero_1)

        statement = select(Hero).where(Hero.name == "Captain North America")
        results = session.exec(statement)
        hero_2 = results.one()
        print("Hero 2:", hero_2)

        hero_1.age = 16
        hero_1.name = "Spider-Youngster"
        session.add(hero_1)

        hero_2.name = "Captain North America Except Canada"
        hero_2.age = 110
        session.add(hero_2)

        session.commit()
        session.refresh(hero_1)
        session.refresh(hero_2)

        print("Updated hero 1:", hero_1)
        print("Updated hero 2:", hero_2)


def delete_heroes():
```

```python
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.one()
        print("Hero: ", hero)

        session.delete(hero)
        session.commit()

        print("Deleted hero:", hero)

        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.first()

        if hero is None:
            print("There's no hero named Spider-Youngster")


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()
    delete_heroes()


if __name__ == "__main__":
    main()
```

As this is a new function `delete_heroes()`, we'll also add it to the `main()` function so that we call it when executing the program from the command line:

Python 3.10+

```python
# Code above omitted 👆

def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()
    delete_heroes()


if __name__ == "__main__":
    main()
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero_1 = results.one()
        print("Hero 1:", hero_1)

        statement = select(Hero).where(Hero.name == "Captain North America")
        results = session.exec(statement)
        hero_2 = results.one()
        print("Hero 2:", hero_2)

        hero_1.age = 16
        hero_1.name = "Spider-Youngster"
        session.add(hero_1)

        hero_2.name = "Captain North America Except Canada"
        hero_2.age = 110
        session.add(hero_2)

        session.commit()
        session.refresh(hero_1)
        session.refresh(hero_2)

        print("Updated hero 1:", hero_1)
        print("Updated hero 2:", hero_2)


def delete_heroes():
```

```python
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.one()
        print("Hero: ", hero)

        session.delete(hero)
        session.commit()

        print("Deleted hero:", hero)

        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.first()

        if hero is None:
            print("There's no hero named Spider-Youngster")


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()
    delete_heroes()


if __name__ == "__main__":
    main()
```

That will print the same existing hero **Spider-Youngster**:

```bash
```

Delete the Hero from the Session

Now, very similar to how we used `session.add()` to add or update new heroes, we can use `session.delete()` to delete the hero from the session:

Python 3.10+

```python
# Code above omitted 👆

def delete_heroes():
```

```python
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.one()
        print("Hero: ", hero)

        session.delete(hero)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero_1 = results.one()
        print("Hero 1:", hero_1)

        statement = select(Hero).where(Hero.name == "Captain North America")
        results = session.exec(statement)
        hero_2 = results.one()
        print("Hero 2:", hero_2)

        hero_1.age = 16
        hero_1.name = "Spider-Youngster"
        session.add(hero_1)

        hero_2.name = "Captain North America Except Canada"
        hero_2.age = 110
        session.add(hero_2)

        session.commit()
        session.refresh(hero_1)
        session.refresh(hero_2)

        print("Updated hero 1:", hero_1)
        print("Updated hero 2:", hero_2)


def delete_heroes():
```

```python
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.one()
        print("Hero: ", hero)

        session.delete(hero)
        session.commit()

        print("Deleted hero:", hero)

        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.first()

        if hero is None:
            print("There's no hero named Spider-Youngster")


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()
    delete_heroes()


if __name__ == "__main__":
    main()
```

Commit the Session

To save the current changes in the session, **commit** it.

This will save all the changes stored in the **session**, like the deleted hero:

Python 3.10+

```python
# Code above omitted 👆

def delete_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.one()
        print("Hero: ", hero)

        session.delete(hero)
        session.commit()

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero_1 = results.one()
        print("Hero 1:", hero_1)

        statement = select(Hero).where(Hero.name == "Captain North America")
        results = session.exec(statement)
        hero_2 = results.one()
        print("Hero 2:", hero_2)

        hero_1.age = 16
        hero_1.name = "Spider-Youngster"
        session.add(hero_1)

        hero_2.name = "Captain North America Except Canada"
        hero_2.age = 110
        session.add(hero_2)

        session.commit()
        session.refresh(hero_1)
        session.refresh(hero_2)

        print("Updated hero 1:", hero_1)
        print("Updated hero 2:", hero_2)


def delete_heroes():
```

```python
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.one()
        print("Hero: ", hero)

        session.delete(hero)
        session.commit()

        print("Deleted hero:", hero)

        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.first()

        if hero is None:
            print("There's no hero named Spider-Youngster")


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()
    delete_heroes()


if __name__ == "__main__":
    main()
```

The same as we have seen before, `.commit()` will also save anything else that was added to the session. Including updates, or created heroes.

This commit after deleting the hero will generate this output:

```bash
```

Print the Deleted Object

Now the hero is deleted from the database.

If we tried to use `session.refresh()` with it, it would raise an exception, because there's no data in the database for this hero.

Nevertheless, the object is still available with its data, but now it's not connected to the session and it no longer exists in the database.

As the object is not connected to the session, it is not marked as "expired", the session doesn't even care much about this object anymore.

Because of that, the object still contains its attributes with the data in it, so we can print it:

Python 3.10+

```
# Code above omitted 👆

def delete_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.one()
        print("Hero: ", hero)

        session.delete(hero)
        session.commit()

        print("Deleted hero:", hero)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero_1 = results.one()
        print("Hero 1:", hero_1)

        statement = select(Hero).where(Hero.name == "Captain North America")
        results = session.exec(statement)
        hero_2 = results.one()
        print("Hero 2:", hero_2)

        hero_1.age = 16
        hero_1.name = "Spider-Youngster"
        session.add(hero_1)

        hero_2.name = "Captain North America Except Canada"
        hero_2.age = 110
        session.add(hero_2)

        session.commit()
        session.refresh(hero_1)
        session.refresh(hero_2)

        print("Updated hero 1:", hero_1)
        print("Updated hero 2:", hero_2)


def delete_heroes():
```

```
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.one()
        print("Hero: ", hero)

        session.delete(hero)
        session.commit()

        print("Deleted hero:", hero)

        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.first()

        if hero is None:
            print("There's no hero named Spider-Youngster")


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()
    delete_heroes()


if __name__ == "__main__":
    main()
```

This will output:

bash

Query the Database for the Same Row

To confirm if it was deleted, now let's query the database again, with the same `"Spider-Youngster"` name:

Python 3.10+

```
# Code above omitted 👆

def delete_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.one()
        print("Hero: ", hero)

        session.delete(hero)
```

```
        session.commit()

        print("Deleted hero:", hero)

        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.first()

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero_1 = results.one()
        print("Hero 1:", hero_1)

        statement = select(Hero).where(Hero.name == "Captain North America")
        results = session.exec(statement)
        hero_2 = results.one()
        print("Hero 2:", hero_2)

        hero_1.age = 16
        hero_1.name = "Spider-Youngster"
        session.add(hero_1)

        hero_2.name = "Captain North America Except Canada"
        hero_2.age = 110
        session.add(hero_2)

        session.commit()
        session.refresh(hero_1)
        session.refresh(hero_2)

        print("Updated hero 1:", hero_1)
        print("Updated hero 2:", hero_2)


def delete_heroes():
```

```python
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.one()
        print("Hero: ", hero)

        session.delete(hero)
        session.commit()

        print("Deleted hero:", hero)

        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.first()

        if hero is None:
            print("There's no hero named Spider-Youngster")


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()
    delete_heroes()


if __name__ == "__main__":
    main()
```

Here we are using `results.first()` to get the first object found (in case it found multiple) or `None`, if it didn't find anything.

If we used `results.one()` instead, it would raise an exception, because it expects exactly one result.

And because we just deleted that hero, this should not find anything and we should get `None`.

This will execute some SQL in the database and output:

```bash
```

Confirm the Deletion

Now let's just confirm that, indeed, no hero was found in the database with that name.

We'll do it by checking that the "first" item in the `results` is `None`:

Python 3.10+

```python
# Code above omitted 👆

def delete_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.one()
        print("Hero: ", hero)

        session.delete(hero)
        session.commit()

        print("Deleted hero:", hero)

        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.first()

        if hero is None:
            print("There's no hero named Spider-Youngster")

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero_1 = results.one()
        print("Hero 1:", hero_1)

        statement = select(Hero).where(Hero.name == "Captain North America")
        results = session.exec(statement)
        hero_2 = results.one()
        print("Hero 2:", hero_2)

        hero_1.age = 16
        hero_1.name = "Spider-Youngster"
        session.add(hero_1)

        hero_2.name = "Captain North America Except Canada"
        hero_2.age = 110
        session.add(hero_2)

        session.commit()
        session.refresh(hero_1)
        session.refresh(hero_2)

        print("Updated hero 1:", hero_1)
        print("Updated hero 2:", hero_2)


def delete_heroes():
```

```python
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.one()
        print("Hero: ", hero)

        session.delete(hero)
        session.commit()

        print("Deleted hero:", hero)

        statement = select(Hero).where(Hero.name == "Spider-Youngster")
        results = session.exec(statement)
        hero = results.first()

        if hero is None:
            print("There's no hero named Spider-Youngster")


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()
    delete_heroes()


if __name__ == "__main__":
    main()
```

This will output:

```bash
```

Review the Code

Now let's review all that code:

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
```

```python
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero_1 = results.one()
        print("Hero 1:", hero_1)

        statement = select(Hero).where(Hero.name == "Captain North America")
        results = session.exec(statement)
        hero_2 = results.one()
        print("Hero 2:", hero_2)

        hero_1.age = 16
        hero_1.name = "Spider-Youngster"
        session.add(hero_1)

        hero_2.name = "Captain North America Except Canada"
        hero_2.age = 110
        session.add(hero_2)

        session.commit()
        session.refresh(hero_1)
        session.refresh(hero_2)

        print("Updated hero 1:", hero_1)
        print("Updated hero 2:", hero_2)


def delete_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Youngster")  1
        results = session.exec(statement)  2
        hero = results.one()  3
        print("Hero: ", hero)  4

        session.delete(hero)  5
        session.commit()  6
```

```
        print("Deleted hero:", hero)  7

        statement = select(Hero).where(Hero.name == "Spider-Youngster")  8
        results = session.exec(statement)  9
        hero = results.first()  10

        if hero is None:  11
            print("There's no hero named Spider-Youngster")  12
    13


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()
    delete_heroes()


if __name__ == "__main__":
    main()
```

1. Select the hero we will delete.
2. Execute the query with the select statement object.

   This generates the output:

   ```
   INFO Engine BEGIN (implicit)
   INFO Engine SELECT hero.id, hero.name, hero.secret_name, hero.age
   FROM hero
   WHERE hero.name = ?
   INFO Engine [no key 0.00011s] ('Spider-Youngster',)
   ```

3. Get one hero object, expecting exactly one.

   > **Tip**
   >
   > This ensures there's no more than one, and that there's exactly one, not `None`.
   >
   > This would never return `None`, instead it would raise an exception.

4. Print the hero object.

   This generates the output:

   ```
   Hero:  name='Spider-Youngster' secret_name='Pedro Parqueador' age=16 id=2
   ```

5. Delete the hero from the **session**.

   This marks the hero as deleted from the session, but it will not be removed from the database until we **commit** the changes.

6. Commit the session.

   This saves the changes in the session, including deleting this row.

   It generates the output:

   ```
   INFO Engine DELETE FROM hero WHERE hero.id = ?
   INFO Engine [generated in 0.00020s] (2,)
   INFO Engine COMMIT
   ```

7. Print the deleted hero object.

   The hero is deleted in the database. And is marked as deleted in the session.

But we still have the object in memory with its data, so we can use it to print it.

This generates the output:

```
Deleted hero: name='Spider-Youngster' secret_name='Pedro Parqueador' age=16 id=2
```

8 Select the same hero again.

We'll do this to confirm if the hero is really deleted.

9 Execute the select statement.

This generates the output:

```
INFO Engine BEGIN (implicit)
INFO Engine SELECT hero.id, hero.name, hero.secret_name, hero.age
FROM hero
WHERE hero.name = ?
INFO Engine [no key 0.00013s] ('Spider-Youngster',)
```

10 Get the "first" item from the `results`.

If no items were found, this will return `None`, which is what we expect.

11 Check if the first item from the results is `None`.

12 If this first item is indeed `None`, it means that it was correctly deleted from the database.

Now we can print a message to confirm.

This generates the output:

```
There's no hero named Spider-Youngster
```

13 This is the end of the `with` block, here the **session** executes its closing code.

This generates the output:

```
INFO Engine ROLLBACK
```

Tip

Check out the number bubbles to see what is done by each line of code.

Recap

To delete rows with **SQLModel** you just have to `.delete()` them with the **session**, and then, as always, `.commit()` the session to save the changes to the database. 🔥